

Debugging CV code — failure modes you'll meet first

A short reference deck. Show this once on Day 1 afternoon, point back to it any time someone hits a bug. The goal is to make "what kind of bug is this?" become a reflex.

The five bugs that eat the most time

1. Wrong tensor shape

Symptom: `RuntimeError: mat1 and mat2 shapes cannot be multiplied` or "expected input of shape (N, 3, H, W) but got (N, H, W, 3)".

Reflex: print `.shape` at every step. Especially before and after `Flatten`, `permute`, `view`, and the first `Linear`.

2. HWC vs CHW

Symptom: image looks like static, or accuracy stuck at random.

Cause: loaded an image as `(H, W, C)` and fed it to a `Conv2d` expecting `(C, H, W)`.

Reflex: if shape `[..., 3]` ends in a small number, you probably have HWC.

Three more, less common but nasty

Normalization mismatch

If you fine-tune a torchvision model, you must use ImageNet mean/std. If you trained from scratch on your dataset, use *its* mean/std. Mixing them silently degrades accuracy.

Augmenting validation

Augmentations belong only on the train set. Augmenting validation makes the metric noisy and uncomparable.

Wrong **Flatten** size

`nn.Linear(in_features, ...)` must match `C * H * W` after the last conv block.

Compute it on paper from the shape rule, or run one batch through and read the shape off **Flatten** output.

The 30-second sanity check

Before training anything:

```
xb, yb = next(iter(train_loader))
xb, yb = xb.to(device), yb.to(device)
logits = model(xb)
print(xb.shape, "->", logits.shape)
print("loss on init:", F.cross_entropy(logits, yb).item())
```

If this runs without error and the loss is roughly $-\log(1/\text{num_classes})$, you're set to train.